

Módulo 6 - Teoría

Curso Spring Boot 2026

Jaime Gallo

Índice

Módulo 6 - Seguridad con Spring Security y JWT	5
Propósito del módulo	5
Qué aprenderás	5
6.1 - Introducción a Spring Security	5
6.1.1 Qué es Spring Security	5
6.1.1.1 Definición y propósito	5
6.1.1.2 El “susto” inicial	6
6.1.1.3 Autenticación vs autorización	6
6.1.2 La cadena de filtros	6
6.1.2.1 Qué es un filtro HTTP	6
6.1.2.2 La cadena de filtros de Spring Security	6
6.1.2.3 Orden de los filtros principales	6
6.1.3 Configuración moderna	7
6.1.3.1 WebSecurityConfigurerAdapter (deprecado)	7
6.1.3.2 SecurityFilterChain como bean	7
6.1.3.3 Métodos principales de la configuración	7
6.1.4 Autenticación en detalle	7
6.1.4.1 AuthenticationManager	7
6.1.4.2 UserDetailsService	7
6.1.4.3 PasswordEncoder	8
6.1.5 Ecosistema de Spring Security	8
6.1.5.1 Autenticación en memoria vs base de datos	8
6.1.5.2 OAuth2 y JWT	8
6.1.5.3 Testing de seguridad	8
Ejemplo mínimo de configuración explícita	8
Profundización: leer la seguridad como un pipeline	9
6.2 - Usuarios en memoria, roles y PasswordEncoder	9
6.2.1 El modelo de usuario en Spring Security	9
6.2.1.1 Qué es un UserDetails	9
6.2.1.2 Qué es un UserDetailsService	9
6.2.1.3 Roles y authorities	9
6.2.2 Usuarios en memoria	10
6.2.2.1 InMemoryUserDetailsManager	10
6.2.2.2 Por qué usar usuarios en memoria	10
6.2.2.3 Definir roles vs authorities	10
6.2.3 Cifrado de contraseñas	10
6.2.3.1 Por qué no se guardan contraseñas en texto plano	10
6.2.3.2 BCryptPasswordEncoder	10
6.2.3.3 DelegatingPasswordEncoder	10
6.2.4 Integración con la cadena de filtros	11

6.2.4.1	Cómo se conecta el UserDetailsService con la autenticación	11
6.2.4.2	HTTP Basic y el flujo de autenticación	11
6.2.4.3	Ver el SecurityContext en el controlador	11
6.2.5	Testing y buenas prácticas	11
6.2.5.1	Probar la autenticación con curl	11
6.2.5.2	Buenas prácticas con usuarios en memoria	11
6.2.5.3	Testing de usuarios en memoria	12
	Ejemplo de usuarios temporales	12
	Profundización: identidad, credenciales y permisos no son lo mismo	12
6.3	- Usuarios en base de datos y registro	12
6.3.1	Por qué usuarios en base de datos	12
6.3.1.1	Las limitaciones de los usuarios en memoria	12
6.3.1.2	Modelo de datos: Usuario y Rol	13
6.3.1.3	@ManyToMany: la relación entre Usuario y Rol	13
6.3.2	UserDetailsService personalizado	13
6.3.2.1	Por qué un UserDetailsService personalizado	13
6.3.2.2	Estructura del UserDetailsService personalizado	13
6.3.2.3	Transformación de roles a authorities	13
6.3.3	Integración con la configuración	13
6.3.3.1	Cómo detecta Spring Security el UserDetailsService	13
6.3.3.2	Configuración de la cadena de filtros	14
6.3.3.3	Inicialización de usuarios	14
6.3.4	Registro de usuarios	14
6.3.4.1	Por qué un endpoint de registro	14
6.3.4.2	DTO de registro	14
6.3.4.3	Servicio de registro	14
6.3.5	Buenas prácticas y tests	15
6.3.5.1	Buenas prácticas con usuarios en base de datos	15
6.3.5.2	Errores comunes	15
6.3.5.3	Testing de usuarios en base de datos	15
	Esqueleto del servicio de identidades	15
	Profundización: diseñar una identidad persistente sin filtrar el modelo interno	15
6.4	- Autorización por URL y por método	16
6.4.1	Autorización por URL vs por método	16
6.4.1.1	Autorización por URL	16
6.4.1.2	Autorización por método	16
6.4.1.3	Cuándo usar cada una	16
6.4.2	Roles y authorities	16
6.4.2.1	hasRole vs hasAuthority	16
6.4.2.2	hasAnyRole y hasAnyAuthority	17
6.4.2.3	Expresiones complejas con SpEL	17
6.4.3	Seguridad por método	17
6.4.3.1	Activar la seguridad por método	17
6.4.3.2	@PreAuthorize en detalle	17
6.4.3.3	@Secured y @PostAuthorize	17
6.4.4	Acceso al usuario autenticado	17
6.4.4.1	Desde el controlador	17
6.4.4.2	Desde el servicio	18
6.4.4.3	Diferencia entre principal y authorities	18
6.4.5	Buenas prácticas y testing	18
6.4.5.1	Buenas prácticas con autorización	18
6.4.5.2	Errores comunes	18
6.4.5.3	Testing de autorización	18
	Ejemplo de reglas combinadas	18

Profundización: autorización como política verificable	19
6.5 - JWT: estructura, firma y ciclo de vida	19
6.5.1 Qué es un JWT	19
6.5.1.1 Definición y origen	19
6.5.1.2 El problema que resuelve	19
6.5.1.3 Stateless vs stateful	19
6.5.2 Estructura del token	20
6.5.2.1 Tres partes separadas por puntos	20
6.5.2.2 El header	20
6.5.2.3 El payload	20
6.5.3 Firma	20
6.5.3.1 Cómo se calcula la firma	20
6.5.3.2 Firma simétrica vs asimétrica	20
6.5.3.3 Por qué la firma garantiza la integridad	20
6.5.4 Casos de uso y ciclo de vida	21
6.5.4.1 Casos de uso ideales	21
6.5.4.2 Cuando no usar JWT	21
6.5.4.3 El ciclo de vida de un token	21
6.5.5 Errores, buenas prácticas y alternativas	21
6.5.5.1 Errores comunes con JWT	21
6.5.5.2 Buenas prácticas con JWT	21
6.5.5.3 Alternativas a JWT	21
Lectura conceptual de un JWT	22
Profundización: qué confianza aporta realmente un JWT	22
6.6 - Generación y validación de JWT con JJWT	22
6.6.1 JJWT	22
6.6.1.1 Qué es JJWT y por qué usarla	22
6.6.1.2 Los módulos de JJWT	22
6.6.1.3 La API de JJWT	22
6.6.2 Clave criptográfica	23
6.6.2.1 Cómo se representa una clave en JJWT	23
6.6.2.2 Generar una clave segura	23
6.6.2.3 Construir la clave en el código	23
6.6.3 JwtService	23
6.6.3.1 La estructura del servicio	23
6.6.3.2 Generar el token de acceso	23
6.6.3.3 Generar el refresh token	23
6.6.4 Login	24
6.6.4.1 El flujo de login	24
6.6.4.2 El AuthenticationManager en el login	24
6.6.4.3 El DTO de respuesta del login	24
6.6.5 Refresh y seguridad operacional	24
6.6.5.1 El endpoint de refresh	24
6.6.5.2 Buenas prácticas con tokens	24
6.6.5.3 Errores comunes	24
Generación con JJWT 0.13.0	25
Profundización: separar criptografía, autenticación y ciclo de tokens	25
6.7 - Filtro JWT y reconstrucción de la identidad	25
6.7.1 El problema de la autenticación con JWT	25
6.7.1.1 El problema de la autenticación con JWT	25
6.7.1.2 Por qué OncePerRequestFilter	25
6.7.1.3 El flujo completo de una petición con JWT	26
6.7.2 Cabecera Authorization	26

6.7.2.1 La cabecera Authorization	26
6.7.2.2 Qué hacer si no hay token	26
6.7.2.3 No usar el filtro para autorizar	26
6.7.3 Authentication y principal	26
6.7.3.1 Qué es un Authentication	26
6.7.3.2 Usar UsuarioPrincipal como principal	26
6.7.3.3 El UsuarioDetailsService en el flujo	27
6.7.4 Registro del filtro	27
6.7.4.1 Añadir el filtro a la cadena	27
6.7.4.2 El filtro como bean	27
6.7.4.3 Orden de los filtros	27
6.7.5 Diagnóstico y buenas prácticas	27
6.7.5.1 Depurar el filtro	27
6.7.5.2 Errores comunes	27
6.7.5.3 Buenas prácticas	28
Esqueleto del filtro	28
Profundización: un filtro robusto hace poco, pero lo hace bien	28
6.8 - Configuración final stateless, errores y autorización completa	28
6.8.1 SecurityFilterChain final	28
6.8.1.1 El SecurityFilterChain con JWT	28
6.8.1.2 Por qué personalizar los manejadores de error	29
6.8.1.3 Estructura del JwtAuthenticationEntryPoint	29
6.8.2 Mapa de endpoints	29
6.8.2.1 Clasificación de endpoints	29
6.8.2.2 Orden de las reglas	29
6.8.2.3 Endpoints con permisos específicos	29
6.8.3 Roles con JWT	29
6.8.3.1 Los roles vienen del token	29
6.8.3.2 hasRole y hasAuthority con JWT	30
6.8.3.3 Combinar filtro, URL y método	30
6.8.4 Flujo completo y logout	30
6.8.4.1 Diagrama del flujo	30
6.8.4.2 Manejo de la expiración en el cliente	30
6.8.4.3 Logout con JWT	30
6.8.5 Diagnóstico y buenas prácticas	30
6.8.5.1 Diagnóstico de problemas comunes	30
6.8.5.2 Errores comunes	31
6.8.5.3 Buenas prácticas	31
Configuración final resumida	31
Profundización: cerrar la arquitectura sin dejar puertas laterales	31
6.9 - Pruebas de seguridad	32
6.9.1 La seguridad es funcionalidad	32
6.9.1.1 La seguridad es funcionalidad	32
6.9.1.2 Qué se puede testear	32
6.9.1.3 Herramientas de testing de seguridad	32
6.9.2 Usuarios simulados	32
6.9.2.1 @WithMockUser	32
6.9.2.2 @WithUserDetails	32
6.9.2.3 @WithAnonymousUser	32
6.9.3 Post-processors	33
6.9.3.1 Post-processors para peticiones	33
6.9.3.2 El post-processor jwt()	33
6.9.3.3 Verificaciones con SecurityMockMvcResultMatchers	33
6.9.4 Tests de flujos	33

6.9.4.1 Tests de autenticación (login)	33
6.9.4.2 Tests de refresh y logout	33
6.9.4.3 Tests de autorización	33
6.9.5 Buenas prácticas de tests	33
6.9.5.1 Buenas prácticas con tests de seguridad	33
6.9.5.2 Errores comunes	34
6.9.5.3 Estructura de un test de seguridad completo	34
Ejemplo con MockMvc	34
Profundización: construir una suite que detecte regresiones reales	34

Cierre del módulo

35

Módulo 6 - Seguridad con Spring Security y JWT

Propósito del módulo

Hasta aquí la aplicación ya dispone de una API REST, persistencia JPA, validación, tratamiento de errores, perfiles, CORS y documentación OpenAPI. M6 incorpora la capa que decide **quién puede entrar** y **qué puede hacer** una vez autenticado. El recorrido comienza con la protección automática de Spring Security, continúa con usuarios y roles, y desemboca en una arquitectura JWT stateless con access token, refresh token, revocación, respuestas 401/403 coherentes y pruebas automatizadas.

El objetivo no es memorizar anotaciones aisladas. Al terminar debes ser capaz de seguir una petición completa: desde la cabecera HTTP, pasando por la cadena de filtros y el SecurityContext, hasta la autorización por URL o por método y la ejecución del controlador. También debes entender qué garantiza realmente la firma de un JWT, cómo se valida un token y por qué una API sin sesión necesita un diseño explícito para refresh y logout.

Qué aprenderás

- Diferenciar autenticación, autorización, 401 y 403.
- Configurar SecurityFilterChain con la API moderna de Spring Security.
- Modelar usuarios, roles y authorities primero en memoria y después en base de datos.
- Aplicar autorización por ruta y por método.
- Entender la estructura, firma y ciclo de vida de un JWT.
- Generar y validar tokens con JWT 0.13.0.
- Integrar un filtro JWT en una aplicación STATELESS.
- Diseñar login, refresh y logout/revocación sin depender de sesión HTTP.
- Verificar el comportamiento con MockMvc, Spring Security Test e integración real.

6.1 - Introducción a Spring Security

6.1.1 Qué es Spring Security

6.1.1.1 Definición y propósito

Spring Security es la infraestructura de seguridad del ecosistema Spring. En una API REST se ocupa de dos responsabilidades que conviene mantener separadas: establecer una identidad autenticada y comprobar si esa identidad puede ejecutar una operación concreta. Esa separación permite razonar con claridad sobre cualquier fallo: primero preguntamos si existe una identidad válida y después si posee los permisos necesarios.

La seguridad se aplica antes de que el controlador atienda la petición. Por eso una ruta puede quedar bloqueada sin tocar una sola línea del controlador. Esta propiedad es útil porque centraliza políticas transversales y evita repetir comprobaciones en cada endpoint.

6.1.1.2 El “susto” inicial

Al añadir `spring-boot-starter-security`, Spring Boot adopta una postura conservadora: las rutas dejan de estar abiertas por defecto. Si todavía no existe una configuración propia, el arranque crea un usuario de desarrollo y una contraseña temporal. Una petición que antes funcionaba de forma anónima empieza a responder con rechazo de autenticación.

Ese cambio no significa que la aplicación se haya roto; significa que el framework ha colocado una barrera delante de la API. En el módulo utilizaremos este comportamiento como experimento inicial y después lo sustituiremos por una configuración explícita y reproducible.

6.1.1.3 Autenticación vs autorización

Autenticar significa demostrar quién eres. **Autorizar** significa decidir qué puede hacer esa identidad. En el flujo que construiremos, un 401 indica que la autenticación no existe o no es válida; un 403 aparece cuando sí hay identidad pero no tiene permiso suficiente.

La diferencia es operativa: frente a un 401 el cliente debe aportar credenciales válidas; frente a un 403 no sirve volver a enviar las mismas credenciales. Esta distinción también guía los tests: debemos comprobar por separado el acceso anónimo, el usuario autenticado sin privilegios y el usuario con el rol correcto.

6.1.2 La cadena de filtros

6.1.2.1 Qué es un filtro HTTP

Un filtro HTTP intercepta la petición antes de que llegue al `DispatcherServlet` y puede observarla, modificarla, rechazarla o dejarla continuar. Los filtros forman una cadena, por lo que el orden importa: una decisión tomada al principio condiciona lo que verán los componentes posteriores.

Spring Security se apoya precisamente en este mecanismo. En lugar de concentrar toda la seguridad en una clase, reparte responsabilidades entre filtros especializados: carga del contexto, autenticación, gestión de excepciones y otras tareas. Entender la cadena evita tratar la seguridad como una “caja negra”.

6.1.2.2 La cadena de filtros de Spring Security

Cuando una petición entra en una aplicación protegida, pasa por una cadena administrada por Spring Security antes de llegar a MVC. Algunos filtros preparan el `SecurityContext`; otros intentan construir un objeto `Authentication`; otros traducen errores de seguridad a respuestas HTTP.

Nuestro filtro JWT se insertará en esa cadena. Su trabajo no será decidir permisos, sino leer un Bearer token válido, reconstruir la identidad y dejarla disponible para las reglas de autorización. Esa separación mantiene el filtro pequeño y hace que las políticas de acceso sigan siendo declarativas.

6.1.2.3 Orden de los filtros principales

El orden es parte del contrato. Si un filtro que autoriza se ejecutara antes de que exista autenticación, vería al cliente como anónimo. Por eso el filtro JWT debe ejecutarse antes del filtro estándar que procesa usuario/contraseña y antes de que las reglas necesiten el `SecurityContext`.

La práctica registrará `JwtAuthenticationFilter` con `addFilterBefore(..., UsernamePasswordAuthenticationFilter.class)`. No se trata de una fórmula arbitraria: expresa que la identidad derivada del token debe estar construida a tiempo para el resto del pipeline.

6.1.3 Configuración moderna

6.1.3.1 WebSecurityConfigurerAdapter (deprecado)

Durante años fue habitual extender `WebSecurityConfigurerAdapter`. Ese enfoque ya no es el recomendado porque ocultaba demasiado estado en herencia y hacía más difícil componer configuraciones. En código actual se prefieren beans explícitos y una configuración funcional de `HttpSecurity`.

Encontrar `WebSecurityConfigurerAdapter` en documentación antigua no significa que el concepto sea inútil, pero sí que debe traducirse a la API moderna. En este curso el estado final nunca dependerá de esa clase obsoleta.

6.1.3.2 SecurityFilterChain como bean

La pieza central de la configuración moderna es un bean `SecurityFilterChain`. Recibe `HttpSecurity`, encadena las decisiones de seguridad y devuelve el resultado con `http.build()`. La configuración queda declarada como una dependencia más del contenedor y puede combinarse con beans como `PasswordEncoder`, manejadores de 401/403 o filtros personalizados.

Este enfoque hace visible qué rutas son públicas, qué sesiones se usan, cómo se integra CORS y dónde se inserta el filtro JWT. La configuración final de M6 será completamente explícita.

6.1.3.3 Métodos principales de la configuración

En `HttpSecurity` aparecen varias áreas que conviene leer como decisiones independientes: `csrf` controla protección CSRF, `cors` integra la política CORS, `sessionManagement` define si habrá sesión, `authorizeHttpRequests` expresa reglas de acceso y `exceptionHandling` decide cómo convertir fallos de seguridad en respuestas HTTP.

La configuración final desactiva CSRF porque usamos una API stateless basada en Bearer tokens, activa CORS, exige STATELESS, abre sólo una allowlist pequeña y protege el resto. Cada llamada responde a una necesidad concreta.

6.1.4 Autenticación en detalle

6.1.4.1 AuthenticationManager

`AuthenticationManager` coordina la autenticación cuando las credenciales son usuario y contraseña. Recibe una solicitud de autenticación y delega en los proveedores configurados. Si las credenciales son válidas devuelve un `Authentication` marcado como autenticado; si no, lanza una excepción.

En M6 lo usaremos en el endpoint de login. El controlador no comparará contraseñas manualmente: delegará en la infraestructura de Spring Security y, sólo después del éxito, generará los tokens JWT.

6.1.4.2 UserDetailsService

`UserDetailsService` responde a una pregunta muy concreta: dado un nombre de usuario, ¿qué datos de seguridad corresponden a esa identidad? Devuelve un `UserDetails` con contraseña codificada y authorities. No autentica por sí mismo; aporta los datos que necesita el proveedor de autenticación.

Primero utilizaremos una implementación en memoria para observar el mecanismo. Después la sustituiremos por una implementación que consulta `UsuarioRepository`, manteniendo el resto del flujo prácticamente igual.

6.1.4.3 PasswordEncoder

Las contraseñas no deben compararse ni almacenarse en texto plano. PasswordEncoder abstrae el algoritmo de codificación y permite a Spring Security comprobar una contraseña presentada contra el hash almacenado.

El proyecto final usa PasswordEncoderFactories.createDelegatingPasswordEncoder(). El prefijo que acompaña al hash indica el algoritmo utilizado y facilita evolucionar la política de contraseñas sin mezclar lógica criptográfica con los servicios de negocio.

6.1.5 Ecosistema de Spring Security

6.1.5.1 Autenticación en memoria vs base de datos

Los usuarios en memoria son excelentes para aprender y para tests pequeños, pero no resuelven persistencia, altas, bajas ni cambios de roles. Una aplicación real necesita identidades almacenadas de forma duradera y una capa que las recupere.

La transición de M6 es deliberada: comenzamos con InMemoryUserDetailsManager porque hace visible el contrato de UserDetails, y lo eliminamos cuando Usuario y Rol ya existen en base de datos.

6.1.5.2 OAuth2 y JWT

JWT y OAuth2 no son sinónimos. JWT es un formato de token firmado; OAuth2 es un marco de autorización con flujos, actores y concesiones. Un sistema OAuth2 puede usar JWT como formato de access token, pero también puede usar tokens opacos.

M6 no construye un servidor OAuth2. Implementa autenticación propia con usuario/contraseña y tokens JWT para comprender los fundamentos de una API stateless. Esta frontera evita atribuir a JWT responsabilidades que pertenecen a un protocolo más amplio.

6.1.5.3 Testing de seguridad

La seguridad debe tratarse como funcionalidad observable. No basta con “ver que compila”: hay que demostrar que una ruta pública responde sin token, que una ruta protegida devuelve 401, que un usuario sin rol recibe 403 y que el rol correcto obtiene éxito.

Spring Security Test ofrece anotaciones y post-processors que permiten construir estos escenarios con MockMvc. En los tests de integración también comprobaremos el flujo real de login, refresh, logout y filtro JWT.

Ejemplo mínimo de configuración explícita

@Bean

```
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(Customizer.withDefaults())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/public/**").permitAll()
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

Este estado es todavía provisional: HTTP Basic se retirará cuando el filtro JWT esté operativo.

Profundización: leer la seguridad como un pipeline

Cuando una petición falla, conviene abandonar la intuición de que “Spring Security ha bloqueado algo” y recorrer el pipeline de manera sistemática. Primero identifica si la URL coincide con una regla pública o protegida. Después comprueba si existe un mecanismo capaz de crear autenticación. A continuación observa qué principal y authorities quedan en el SecurityContext. Por último revisa la regla de autorización concreta. Este orden evita modificar al azar controladores, filtros y configuración.

También es importante entender que seguridad y MVC se tocan, pero no son la misma capa. Un @GetMapping puede estar perfectamente declarado y, aun así, no ejecutarse nunca porque la cadena de seguridad responde antes. Del mismo modo, una excepción de autenticación no debería tratarse como una excepción de negocio del controlador. Por eso el módulo termina usando manejadores específicos de seguridad y conserva @RestControllerAdvice para el resto de la aplicación.

Preguntas de control

- ¿Qué componente decide si una petición anónima puede llegar a un controlador?
- ¿Por qué un 403 implica que la identidad ya existe?
- ¿Qué diferencia hay entre una regla HTTP y una regla de negocio?
- ¿Por qué el orden de filtros y matchers forma parte del comportamiento observable?

6.2 - Usuarios en memoria, roles y PasswordEncoder

6.2.1 El modelo de usuario en Spring Security

6.2.1.1 Qué es un UserDetails

UserDetails es la representación de seguridad de una identidad. Contiene nombre de usuario, contraseña codificada, authorities y varios indicadores de estado de la cuenta. Spring Security trabaja con este contrato en lugar de depender directamente de nuestra entidad JPA.

Separar la entidad de dominio de la representación de seguridad evita acoplar toda la aplicación a una interfaz del framework. En M6 terminaremos utilizando UsuarioPrincipal como adaptador entre Usuario y UserDetails.

6.2.1.2 Qué es un UserDetailsService

UserDetailsService carga un UserDetails por nombre. Su método principal, loadUserByUsername, debe devolver la identidad o lanzar UsernameNotFoundException. La interfaz no sabe si los datos vienen de memoria, JPA, LDAP u otro sistema.

Esa neutralidad permite sustituir la implementación sin cambiar el login. Durante 6.2 la fuente de datos será una colección en memoria; en 6.3 la misma operación consultará la base de datos.

6.2.1.3 Roles y authorities

Spring Security toma decisiones sobre GrantedAuthority. Un rol es una convención construida sobre authorities cuyo nombre suele llevar el prefijo ROLE_. Por eso hasRole("ADMIN") busca internamente ROLE_ADMIN, mientras hasAuthority("ADMIN") exige exactamente esa cadena.

Conviene fijar una única convención. En el proyecto almacenaremos roles con nombres como ROLE_USER, ROLE_GESTOR y ROLE_ADMIN, y los transformaremos directamente en authorities.

6.2.2 Usuarios en memoria

6.2.2.1 InMemoryUserDetailsManager

InMemoryUserDetailsManager almacena objetos UserDetails dentro del proceso. Es rápido de configurar y permite experimentar con varios usuarios y roles sin crear todavía tablas ni repositorios.

Su limitación es precisamente esa: el estado vive en memoria. Al reiniciar, sólo existen los usuarios declarados en configuración. Por eso lo usaremos como etapa didáctica y lo eliminaremos antes del estado final.

6.2.2.2 Por qué usar usuarios en memoria

Una etapa intermedia simple reduce variables. Si queremos entender por qué un ADMIN entra y un USER recibe 403, es más fácil comenzar con usuarios estáticos que introducir a la vez JPA, relaciones muchos-a-muchos y carga de authorities.

El valor de la técnica es aislar el mecanismo. Una vez comprobado, trasladaremos las identidades a base de datos sin cambiar la semántica de roles ni la forma de autorizar.

6.2.2.3 Definir roles vs authorities

Al crear un usuario podemos utilizar `.roles("ADMIN")` o `.authorities("ROLE_ADMIN")`. La primera forma añade el prefijo `ROLE_`; la segunda utiliza literalmente los valores suministrados.

Mezclar ambas sin conocer la diferencia genera fallos difíciles de ver: el usuario parece tener un permiso, pero la expresión busca otro nombre. En el módulo verificaremos las authorities reales que llegan al `SecurityContext`.

6.2.3 Cifrado de contraseñas

6.2.3.1 Por qué no se guardan contraseñas en texto plano

Una base de datos comprometida no debe revelar las contraseñas originales. Por eso se almacenan hashes calculados con algoritmos diseñados para contraseñas, deliberadamente costosos y con sal aleatoria. El objetivo no es “cifrar y descifrar”, sino verificar una entrada sin recuperar el secreto original.

Guardar texto plano o usar hashes rápidos como SHA-256 sin un esquema específico de password hashing es una mala práctica. Spring Security nos obliga además a declarar cómo se codifican las credenciales.

6.2.3.2 BCryptPasswordEncoder

BCrypt incorpora sal y un factor de coste. Dos usuarios con la misma contraseña pueden terminar con hashes distintos, y matches puede comprobar ambos correctamente. El coste configurable dificulta ataques masivos de fuerza bruta.

No compararemos strings de hashes de forma manual. El servicio de registro llamará a `PasswordEncoder.encode(...)` y el proveedor de autenticación utilizará el mismo encoder al validar el login.

6.2.3.3 DelegatingPasswordEncoder

DelegatingPasswordEncoder guarda un identificador de algoritmo junto al hash, por ejemplo `{bcrypt}`. Esto permite que una aplicación reconozca hashes creados con esquemas distintos y pueda migrarlos gradualmente.

Para un curso acumulativo es una opción especialmente útil: mantiene la configuración flexible sin obligarnos a codificar BCrypt directamente en cada clase. El bean de PasswordEncoder será compartido por registro y autenticación.

6.2.4 Integración con la cadena de filtros

6.2.4.1 Cómo se conecta el UserDetailsService con la autenticación

Cuando llega una autenticación de usuario/contraseña, Spring Security busca un proveedor capaz de procesarla. Ese proveedor carga el usuario mediante UserDetailsService, compara la contraseña con PasswordEncoder y, si todo es correcto, crea un Authentication autenticado con sus authorities.

El controlador no necesita conocer estos detalles. Esa separación nos permitirá reutilizar exactamente el mismo mecanismo desde el login JWT de 6.6.

6.2.4.2 HTTP Basic y el flujo de autenticación

HTTP Basic transporta usuario:password codificado en Base64 en cada petición. Base64 no es cifrado, así que sólo debe utilizarse sobre HTTPS. En M6 se usa como herramienta temporal para observar el flujo de autenticación antes de introducir JWT.

Cuando el filtro Basic acepta las credenciales, el SecurityContext queda poblado para el resto de la petición. Más adelante retiraremos HTTP Basic y el contexto será creado a partir del Bearer token.

6.2.4.3 Ver el SecurityContext en el controlador

Una vez autenticado, Spring permite acceder a la identidad mediante Authentication, Principal o anotaciones como @AuthenticationPrincipal. El objeto contiene el principal y las authorities que se utilizarán en autorización.

Consultar el propio perfil es una buena forma de comprobar el resultado del pipeline: la respuesta debe reflejar username, email y roles obtenidos de la identidad actual, no valores enviados por el cliente.

6.2.5 Testing y buenas prácticas

6.2.5.1 Probar la autenticación con curl

curl permite inspeccionar códigos, cabeceras y cuerpo sin depender de una interfaz gráfica. En la etapa Basic usaremos -u usuario:password; en la etapa JWT enviaremos Authorization: Bearer <token>.

La prueba correcta incluye al menos tres casos: sin credenciales, credenciales erróneas y credenciales válidas. Sólo el tercero debe alcanzar el recurso protegido.

6.2.5.2 Buenas prácticas con usuarios en memoria

Los usuarios en memoria deben considerarse configuración temporal o de test. No conviene poner contraseñas reales en el repositorio ni conservar este mecanismo como solución de producción si la aplicación necesita gestión de identidades.

Al terminar 6.2 habrá una restauración explícita: la configuración en memoria se elimina cuando 6.3 introduce UsuarioDetailsService respaldado por JPA.

6.2.5.3 Testing de usuarios en memoria

Un test de seguridad puede declarar usuarios simulados sin depender de la configuración en memoria. `@WithMockUser` resulta útil para probar autorización; si necesitamos verificar la carga real de usuarios, conviene usar tests que involucren el `UserDetailsService` efectivo.

La idea clave es distinguir test de política de acceso y test de autenticación real. Ambos son necesarios, pero responden a preguntas distintas.

Ejemplo de usuarios temporales

`@Bean`

```
UserDetailsService userDetailsService(PasswordEncoder encoder) {
    UserDetails ana = User.withUsername("ana")
        .password(encoder.encode("ana123"))
        .roles("USER")
        .build();
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN")
        .build();
    return new InMemoryUserDetailsManager(ana, admin);
}
```

Este bean existe sólo durante la etapa en memoria y desaparece al pasar a usuarios persistidos.

Profundización: identidad, credenciales y permisos no son lo mismo

Un usuario de seguridad reúne varias piezas que a menudo se mezclan: un identificador, una credencial que permite demostrar la identidad y un conjunto de permisos. La contraseña es una credencial; no es la identidad. Un rol es un permiso; tampoco es la identidad. Esta separación explica por qué Spring Security modela `UserDetails`, `PasswordEncoder` y `GrantedAuthority` como contratos diferentes.

Durante el desarrollo es útil inspeccionar el objeto `Authentication`. Antes de validar credenciales puede contener sólo `username` y `password`; después del éxito contiene un principal estable y `authorities`. Esa transición hace visible qué ha comprobado realmente el framework. Si una prueba de autorización falla, mirar primero las `authorities` evita atribuir el problema a la contraseña.

El almacenamiento de contraseñas merece una precaución adicional. Un hash de contraseña no es un secreto equivalente a la contraseña original, pero sigue siendo información sensible: si se filtra, permite ataques offline. Por eso un coste adecuado, sales automáticas y políticas de cambio siguen siendo importantes aunque nunca almacenemos texto plano.

Preguntas de control

- ¿Por qué `.roles("ADMIN")` no es idéntico a `.authorities("ADMIN")`?
- ¿Qué función cumple `PasswordEncoder.matches` que no cumple una comparación de strings?
- ¿Qué se pierde al reiniciar una aplicación que sólo usa `InMemoryUserDetailsManager`?

6.3 - Usuarios en base de datos y registro

6.3.1 Por qué usuarios en base de datos

6.3.1.1 Las limitaciones de los usuarios en memoria

Una colección en memoria no permite administrar identidades de forma persistente. No hay altas duraderas, cambios de contraseña fiables, asignación de roles desde datos ni continuidad entre reinicios. Tampoco podemos consultar usuarios desde otras partes del dominio.

Por eso 6.3 traslada el modelo a JPA. La autenticación seguirá consumiendo `UserDetails`, pero esos datos se construirán a partir de entidades `Usuario` y `Rol`.

6.3.1.2 Modelo de datos: Usuario y Rol

`Usuario` representa una cuenta autenticable: nombre, email, password codificado, estado y roles. `Rol` representa una autoridad reutilizable. Separarlos evita duplicar cadenas de permisos en cada fila de usuario y permite asignar varias autoridades a una misma identidad.

Los nombres de rol se almacenan con la convención `ROLE_*`, de manera que la conversión a `GrantedAuthority` sea directa y consistente.

6.3.1.3 @ManyToMany: la relación entre Usuario y Rol

Una cuenta puede tener varios roles y un mismo rol puede pertenecer a muchas cuentas. JPA modela esta situación con `@ManyToMany` y una tabla intermedia. La colección de roles debe inicializarse y mantenerse de forma predecible para evitar nulos o duplicados.

No necesitamos serializar la relación como JSON para autenticar. La entidad se usa internamente; hacia el exterior expondremos DTOs que contienen sólo los datos necesarios.

6.3.2 UserDetailsService personalizado

6.3.2.1 Por qué un UserDetailsService personalizado

La implementación personalizada conecta Spring Security con nuestro repositorio. Dado un `username`, busca `Usuario`; si no existe, informa de identidad desconocida; si existe, lo transforma a un principal con password codificado y `authorities`.

La clase no debe implementar lógica de login ni generar tokens. Su responsabilidad es pequeña y reutilizable: cargar una identidad de seguridad desde la fuente de datos.

6.3.2.2 Estructura del UserDetailsService personalizado

Una implementación típica depende de `UsuarioRepository`, ejecuta una consulta como `findByUsername` y devuelve `UsuarioPrincipal`. Si la relación de roles es lazy, la consulta o el contexto transaccional deben garantizar que las `authorities` estén disponibles cuando se construye el principal.

Mantener esta transformación en un único lugar evita que controladores y servicios repitan cómo se interpreta un usuario JPA como identidad de Spring Security.

6.3.2.3 Transformación de roles a authorities

Cada `Rol.nombre` se convierte en `SimpleGrantedAuthority`. Si el rol se almacena como `ROLE_ADMIN`, esa misma cadena debe llegar al `Authentication`.

La coherencia de nombres es crítica: `hasRole("ADMIN")` y `hasAuthority("ROLE_ADMIN")` deben producir el mismo resultado si la `authority` está bien formada. Los tests de autorización comprobarán esta equivalencia práctica.

6.3.3 Integración con la configuración

6.3.3.1 Cómo detecta Spring Security el UserDetailsService

Cuando el contexto contiene un único bean `UserDetailsService` y un `PasswordEncoder` compatible, Spring Boot puede construir el proveedor de autenticación necesario. Aun así, el login explícito

obtendrá `AuthenticationManager` desde `AuthenticationConfiguration`, evitando crear gestores manualmente.

La configuración final no declara usuarios estáticos. La presencia del servicio JPA es suficiente para que la autenticación usuario/contraseña use la base de datos.

6.3.3.2 Configuración de la cadena de filtros

La cadena HTTP no necesita saber cómo se persisten usuarios. Su trabajo es definir rutas, sesión, filtros y manejadores. Esta separación permite evolucionar el almacenamiento de identidades sin reescribir la política HTTP.

En 6.3 todavía podemos conservar HTTP Basic para comprobar la nueva fuente de usuarios. Más adelante será sustituido por JWT, pero `UserDetailsService` seguirá siendo útil para login y reconstrucción del principal.

6.3.3.3 Inicialización de usuarios

Para poder probar el módulo necesitamos roles y usuarios conocidos en el perfil de desarrollo. El inicializador debe ser **idempotente**: si un rol o usuario ya existe, no debe crear duplicados ni cambiar contraseñas de forma inesperada en cada arranque.

La idempotencia convierte los datos de desarrollo en una herramienta reproducible. Además, las contraseñas se codifican con el mismo `PasswordEncoder` usado por la autenticación.

6.3.4 Registro de usuarios

6.3.4.1 Por qué un endpoint de registro

Un sistema con usuarios persistidos necesita un camino controlado para crear cuentas. El endpoint de registro recibe datos públicos, valida formato y unicidad, codifica la contraseña y asigna únicamente los roles permitidos por la política de la aplicación.

Nunca debe aceptar directamente una entidad `Usuario` desde JSON, porque eso permitiría al cliente manipular campos internos como roles, estado o hash de contraseña.

6.3.4.2 DTO de registro

El DTO de registro define la frontera HTTP: username, email y contraseña con restricciones de validación. Su propósito es separar el contrato de entrada del modelo persistente. Incluso si ambos comparten campos, no son la misma responsabilidad.

Las validaciones sintácticas pertenecen al DTO; las reglas que requieren consultar la base de datos, como "username único", pertenecen al servicio.

6.3.4.3 Servicio de registro

`AuthService` coordina la operación: comprueba duplicados, obtiene el rol inicial, codifica la contraseña y persiste la cuenta. El rol por defecto debe ser una decisión de servidor, no un campo de libre elección del cliente.

Los errores de negocio se traducen a respuestas consistentes mediante el manejo de excepciones del proyecto. Así el registro se integra con el contrato de errores heredado de M5.

6.3.5 Buenas prácticas y tests

6.3.5.1 Buenas prácticas con usuarios en base de datos

Las entidades de autenticación contienen información sensible. No deben serializar el hash de contraseña ni exponerse de forma accidental. Los endpoints devuelven DTOs y las consultas se diseñan para cargar sólo lo necesario.

También conviene mantener separadas las operaciones administrativas de las operaciones de perfil. Un usuario puede consultar su propia identidad sin adquirir privilegios para listar o modificar cuentas ajenas.

6.3.5.2 Errores comunes

Los fallos habituales incluyen guardar la contraseña sin codificar, mezclar `ROLE_ADMIN` con `ADMIN`, crear usuarios iniciales duplicados, devolver la entidad `Usuario` completa o confiar en un rol enviado por el cliente durante el registro.

Otro error sutil es dejar activo el `InMemoryUserDetailsManager` al introducir JPA: dos fuentes de identidad pueden generar comportamientos difíciles de explicar. La transición debe ser explícita.

6.3.5.3 Testing de usuarios en base de datos

Los tests deben comprobar tanto persistencia como seguridad. Necesitamos demostrar que el registro crea una cuenta con password codificado, que el login usa esos datos y que las authorities recuperadas de la base de datos gobiernan el acceso.

También conviene verificar los casos negativos: usuario duplicado, email duplicado, credenciales erróneas y ausencia del rol requerido. Un test útil no sólo confirma el “camino feliz”.

Esqueleto del servicio de identidades

```
@Service
public class UsuarioDetailsService implements UserDetailsService {
    private final UsuarioRepository repository;

    public UsuarioDetailsService(UsuarioRepository repository) {
        this.repository = repository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        Usuario usuario = repository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));
        return UsuarioPrincipal.from(usuario);
    }
}
```

Profundización: diseñar una identidad persistente sin filtrar el modelo interno

La entidad `Usuario` pertenece a persistencia, mientras que el JSON de registro y la identidad de Spring Security pertenecen a fronteras distintas. Reutilizar la entidad en todas partes parece ahorrar clases, pero introduce riesgos: un cliente podría enviar campos que no debería controlar, un cambio de esquema podría romper el contrato HTTP y la contraseña codificada podría terminar serializada por error.

Los DTOs y `UsuarioPrincipal` resuelven problemas diferentes. El DTO limita qué entra y qué sale de la API. El principal adapta la entidad al contrato de seguridad y concentra la conversión de roles a authorities. Esta arquitectura permite evolucionar cada capa de manera independiente.

La inicialización de datos de desarrollo también debe diseñarse con cuidado. Un inicializador idempotente consulta antes de crear y puede ejecutarse muchas veces sin multiplicar roles o usuarios. Esto resulta especialmente importante en un curso acumulativo, donde cada módulo arranca repetidamente durante pruebas. La idempotencia no es un detalle cosmético: evita estados que sólo funcionan después de borrar la base de datos.

Preguntas de control

- ¿Por qué el endpoint de registro no acepta un conjunto arbitrario de roles?
- ¿Qué ventaja aporta un `UsuarioPrincipal` frente a exponer directamente `Usuario`?
- ¿Qué fallo produciría mantener simultáneamente usuarios en memoria y usuarios JPA sin una política clara?

6.4 - Autorización por URL y por método

6.4.1 Autorización por URL vs por método

6.4.1.1 Autorización por URL

Las reglas de URL se expresan en `SecurityFilterChain` y se evalúan antes de que el controlador ejecute su lógica. Son adecuadas para políticas transversales fáciles de describir por método HTTP y patrón de ruta: por ejemplo, permitir un endpoint público o exigir ADMIN para una zona administrativa.

Su ventaja principal es la visibilidad: una revisión de `SecurityConfig` permite entender gran parte del perímetro de la API. La desventaja es que no siempre capturan reglas ligadas a argumentos concretos o a decisiones de negocio.

6.4.1.2 Autorización por método

La seguridad por método se aplica sobre métodos de controlador o, preferiblemente, de servicio mediante anotaciones como `@PreAuthorize`. Permite expresar condiciones más cercanas al caso de uso y utilizar información del usuario autenticado o de los parámetros.

Para que funcione hay que activar `@EnableMethodSecurity`. Una vez activada, Spring crea proxies que interceptan la llamada al método y evalúan la expresión antes de ejecutar la lógica protegida.

6.4.1.3 Cuándo usar cada una

URL y método no son enfoques rivales. La configuración HTTP puede establecer el perímetro grueso y la capa de servicio aplicar reglas de dominio más específicas. Esa defensa en profundidad evita que un refactor de rutas elimine accidentalmente una restricción importante.

La regla práctica es colocar la política donde mejor expresa su intención y evitar duplicaciones sin propósito. Si la condición depende del propietario de un recurso, suele pertenecer al servicio; si depende del tipo de endpoint, suele ser clara en la cadena HTTP.

6.4.2 Roles y authorities

6.4.2.1 `hasRole` vs `hasAuthority`

`hasRole("ADMIN")` aplica la convención `ROLE_` y busca `ROLE_ADMIN`. `hasAuthority("ROLE_ADMIN")` compara exactamente la cadena indicada. Ambas expresiones pueden representar la misma política, pero sólo si los nombres almacenados y las authorities construidas siguen la misma convención.

Elegir una convención estable reduce errores. En M6 los roles persistidos usan `ROLE_*`, por lo que las dos formas son equivalentes cuando se emplean correctamente.

6.4.2.2 `hasAnyRole` y `hasAnyAuthority`

Cuando una operación admite varias autoridades, `hasAnyRole("GESTOR", "ADMIN")` es más expresivo que duplicar reglas. La versión `hasAnyAuthority` ofrece la misma idea sin aplicar prefijos.

Estas expresiones son especialmente útiles en endpoints de gestión donde varias categorías de usuario comparten una capacidad. Los tests deben cubrir al menos un rol permitido y uno rechazado.

6.4.2.3 Expresiones complejas con SpEL

`@PreAuthorize` puede usar SpEL para combinar roles, parámetros y datos del principal. Es posible escribir condiciones como "ADMIN o el propio usuario". Esta potencia debe usarse con moderación: una expresión demasiado larga se vuelve difícil de leer y de testear.

Cuando la decisión exige cargar datos o evaluar reglas de negocio, suele ser mejor delegar en un componente de autorización o en el servicio y mantener la expresión corta.

6.4.3 Seguridad por método

6.4.3.1 Activar la seguridad por método

La anotación `@EnableMethodSecurity` habilita la interceptación de métodos protegidos. Sin ella, `@PreAuthorize` puede compilar y quedar visualmente presente sin producir ningún efecto, un fallo especialmente peligroso porque parece que la aplicación está protegida.

Por eso la activación y los tests forman una pareja inseparable: un test de acceso denegado confirma que el proxy de seguridad está realmente operativo.

6.4.3.2 `@PreAuthorize` en detalle

`@PreAuthorize` evalúa la expresión antes de entrar en el método. Puede consultar `authentication`, `authorities`, parámetros por nombre y propiedades del principal. Si la expresión es falsa, Spring corta la ejecución y produce un error de acceso.

En el proyecto lo utilizaremos para proteger operaciones administrativas y para demostrar condiciones que no se expresan cómodamente sólo por URL.

6.4.3.3 `@Secured` y `@PostAuthorize`

`@Secured` ofrece una forma más simple basada principalmente en roles. `@PostAuthorize` evalúa después de ejecutar el método y puede inspeccionar el resultado, aunque implica que la lógica ya se ha ejecutado.

M6 se centra en `@PreAuthorize` porque es flexible y evita realizar trabajo que luego será rechazado. Conocer las alternativas ayuda a reconocer código heredado y a elegir la herramienta adecuada.

6.4.4 Acceso al usuario autenticado

6.4.4.1 Desde el controlador

Un controlador puede recibir `Authentication`, `Principal` o `@AuthenticationPrincipal`. Esto resulta útil para endpoints como "mi perfil", donde la identidad forma parte directa del contrato HTTP.

Aun así, el controlador no debería contener decisiones de negocio complejas. Lo normal es extraer la identidad y delegar en el servicio, manteniendo el controlador como adaptador web.

6.4.4.2 Desde el servicio

En la capa de servicio puede consultarse el `SecurityContextHolder`, aunque conviene encapsular ese acceso para no propagar dependencias de seguridad por todo el dominio. Otra opción es pasar al servicio el identificador del usuario obtenido en la capa web.

La decisión depende del diseño, pero la regla sigue siendo la misma: no confiar en un `userId` enviado por el cliente para representar la identidad autenticada.

6.4.4.3 Diferencia entre principal y authorities

El principal describe **quién** está autenticado; las authorities describen **qué permisos** posee. Mezclar ambos conceptos conduce a expresiones frágiles, por ejemplo asumir que el `username` implica un rol.

`UsuarioPrincipal` permite reunir identificador, `username`, `email` y `authorities` en una representación explícita. Las reglas de autorización leen los permisos; los casos de uso de perfil pueden leer la identidad.

6.4.5 Buenas prácticas y testing

6.4.5.1 Buenas prácticas con autorización

Las reglas deben ser lo más pequeñas y declarativas posible, con denegación por defecto. Las rutas públicas se enumeran de forma explícita y el resto exige autenticación. Los roles administrativos no se aceptan desde el cliente ni se infieren a partir de parámetros.

También es importante revisar el orden de los matchers: una regla amplia colocada antes puede eclipsar otra más específica.

6.4.5.2 Errores comunes

Los errores típicos son confundir `ROLE_ADMIN` con `ADMIN`, olvidar `@EnableMethodSecurity`, usar patrones de URL demasiado amplios, confiar en datos del `request` para decidir identidad o dejar una ruta sensible bajo `permitAll`.

Otro fallo frecuente es probar sólo con el usuario autorizado. Un test negativo revela rápidamente si una regla realmente protege el recurso.

6.4.5.3 Testing de autorización

Una matriz de tests pequeña pero sistemática funciona mejor que casos aislados: anónimo, `USER`, `GESTOR` y `ADMIN` frente a operaciones públicas, autenticadas y administrativas. `@WithMockUser` permite expresar esta matriz con claridad.

Para reglas que dependen de un principal real, los tests de integración complementan los mocks y verifican que los roles persistidos llegan correctamente al `SecurityContext`.

Ejemplo de reglas combinadas

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/api/v1/public/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
```

```
@PreAuthorize("hasRole('ADMIN')")
public void cambiarRoles(Long usuarioId, Set<String> roles) { ... }
```

Profundización: autorización como política verificable

Una política de autorización útil se puede expresar como una matriz: operación, identidad requerida, authorities admitidas y resultado esperado. Esa matriz es más fácil de revisar que una colección dispersa de anotaciones. Al diseñar una nueva ruta, conviene decidir primero en qué categoría cae y después implementar la regla.

La seguridad por URL funciona muy bien para fronteras estables: zonas públicas, administración o gestión. La seguridad por método aporta precisión cuando la decisión depende del caso de uso. Por ejemplo, una regla “ADMIN o el propio usuario” está más cerca de la operación de servicio que del patrón de ruta. Sin embargo, introducir consultas a base de datos dentro de expresiones SpEL complejas suele reducir legibilidad. En esos casos es preferible encapsular la decisión.

La defensa en profundidad no significa repetir exactamente la misma regla en todas partes. Significa que cada capa protege su frontera: HTTP evita exposición accidental de una zona y el servicio garantiza la regla de negocio aunque sea invocado desde otro adaptador.

Preguntas de control

- ¿Qué ocurriría si permitAll() para una ruta amplia apareciera antes de un matcher administrativo?
- ¿Cuándo una condición debe salir de SpEL y convertirse en código explícito?
- ¿Por qué un test con sólo ADMIN no demuestra que USER esté correctamente bloqueado?

6.5 - JWT: estructura, firma y ciclo de vida

6.5.1 Qué es un JWT

6.5.1.1 Definición y origen

JWT significa JSON Web Token y define una representación compacta de claims que puede firmarse digitalmente. Su forma habitual es una cadena de tres segmentos codificados en Base64URL y separados por puntos.

El token no es una sesión escondida ni un formato de cifrado por defecto. Es un contenedor firmado cuya utilidad depende de qué claims contiene, cómo se protege la clave y cómo se valida en cada petición.

6.5.1.2 El problema que resuelve

En una API distribuida resulta útil transportar una prueba verificable de autenticación sin consultar una sesión de servidor en cada llamada. El emisor genera un token y los componentes que conocen la clave o la clave pública pueden verificarlo.

Esto reduce dependencia de estado de sesión, pero traslada responsabilidades al diseño del token: expiración corta, gestión de claves, refresh y revocación cuando sea necesaria.

6.5.1.3 Stateless vs stateful

En un sistema stateful el servidor asocia al cliente con una sesión almacenada. En el enfoque stateless de M6 cada petición aporta la credencial necesaria y SessionCreationPolicy.STATELESS impide depender de HttpSession para conservar autenticación.

“Stateless” no significa “sin ningún estado en todo el sistema”. Por ejemplo, una lista de refresh tokens usados o tokens revocados introduce un estado controlado para resolver logout y rotación.

6.5.2 Estructura del token

6.5.2.1 Tres partes separadas por puntos

Un JWT firmado suele tener `header.payload.signature`. Header y payload son JSON codificados en Base64URL; la firma se calcula sobre esos dos segmentos. Separar visualmente las partes ayuda a entender que sólo la tercera aporta autenticidad.

Decodificar los dos primeros segmentos es trivial y no demuestra que el token sea válido. Cualquier cliente puede leerlos o construir otros distintos.

6.5.2.2 El header

El header describe metadatos del token, principalmente el tipo y el algoritmo de firma. En un token HMAC puede indicar HS256. El validador no debe aceptar algoritmos arbitrarios enviados por un atacante; la política criptográfica pertenece al servidor.

El header es pequeño, pero forma parte exacta de los datos firmados. Cambiarlo invalida la firma.

6.5.2.3 El payload

El payload contiene claims. Algunos son registrados, como `sub`, `iat` y `exp`; otros son propios de la aplicación, por ejemplo roles. Los claims no deben incluir secretos, porque el payload se puede decodificar sin conocer la clave.

En M6 `sub` identifica al usuario y la expiración limita la vida del access token. Las authorities se reconstruyen de forma controlada a partir de la identidad del sistema.

6.5.3 Firma

6.5.3.1 Cómo se calcula la firma

Con HMAC, el emisor combina el header y payload codificados, calcula un MAC con una clave secreta y codifica el resultado en Base64URL. El receptor repite el cálculo con la misma clave y compara el resultado de manera segura.

Si cambia un solo byte del contenido, el cálculo ya no coincide. Por eso modificar manualmente el payload produce un token que se puede leer pero no aceptar.

6.5.3.2 Firma simétrica vs asimétrica

HS256 usa una clave simétrica: quien firma y quien verifica conocen el mismo secreto. Algoritmos como RS256 usan una clave privada para firmar y una pública para verificar.

La elección depende de la arquitectura. Para una aplicación docente monolítica, HMAC simplifica el ejemplo. En sistemas con muchos verificadores independientes, la criptografía asimétrica puede facilitar la distribución de claves.

6.5.3.3 Por qué la firma garantiza la integridad

La firma vincula criptográficamente el contenido con una clave. Si un atacante cambia roles, `sub` o expiración sin poder recalcular una firma válida, la verificación falla.

La firma no oculta la información. Integridad y confidencialidad son propiedades distintas. Si un claim no debe ser visible al cliente, no debe colocarse en un JWT firmado sin cifrado adicional.

6.5.4 Casos de uso y ciclo de vida

6.5.4.1 Casos de uso ideales

JWT encaja bien cuando una API necesita credenciales portables, vida limitada y validación local. Es habitual en APIs REST, arquitecturas distribuidas y comunicación entre componentes que comparten una política de confianza.

La ganancia principal es desacoplar la validación de una sesión central. A cambio hay que gestionar expiraciones y renovación con cuidado.

6.5.4.2 Cuándo no usar JWT

No todo sistema necesita JWT. Una aplicación web monolítica con sesión de servidor puede ser más simple y más fácil de revocar. Tampoco conviene usar JWT como almacén de datos de usuario ni crear tokens de vida muy larga para evitar implementar refresh.

La tecnología debe resolver un problema real. En M6 se usa porque queremos aprender una API REST stateless, no porque sea obligatoria en Spring Boot.

6.5.4.3 El ciclo de vida de un token

El access token debe tener vida relativamente corta. Cuando expira, el cliente utiliza un refresh token válido para obtener un nuevo par. El refresh token tiene una vida mayor y requiere controles adicionales porque su compromiso permite renovar credenciales.

Nuestro diseño rota el refresh token y considera de un solo uso los tokens registrados. El logout revoca los tokens que el servidor decide recordar.

6.5.5 Errores, buenas prácticas y alternativas

6.5.5.1 Errores comunes con JWT

Los fallos más graves son aceptar un token sólo porque se puede decodificar, utilizar una clave demasiado corta, no verificar expiración, incluir datos sensibles, reutilizar refresh tokens indefinidamente o confundir un refresh token con un access token.

También es peligroso confiar ciegamente en claims de roles si el sistema no define cómo se sincronizan con los permisos actuales.

6.5.5.2 Buenas prácticas con JWT

Las claves deben estar externalizadas, ser suficientemente largas y rotarse con un procedimiento definido. Los access tokens deben caducar, la verificación debe fijar el algoritmo esperado y los errores no deben revelar secretos.

En producción la clave no se incluye con fallback inseguro en properties. M6 exige `jwt.secret=${JWT_SECRET}` para el perfil de producción.

6.5.5.3 Alternativas a JWT

Las sesiones de servidor siguen siendo una opción válida. También existen tokens opacos, donde el servidor de recursos consulta o introspecciona el token, y soluciones estándar OAuth2/OpenID Connect que delegan identidad y autorización en componentes especializados.

Conocer estas alternativas ayuda a no convertir JWT en una solución universal. Nuestro objetivo es comprender el mecanismo y sus límites.

Lectura conceptual de un JWT

eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbmEiLCJleHAiOjE3MDAwMDAwMDB9.<firma>

Los dos primeros segmentos se pueden decodificar. Sólo una verificación criptográfica de la firma y de claims como exp permite confiar en el token.

Profundización: qué confianza aporta realmente un JWT

Un JWT firmado permite confiar en que el contenido verificado fue emitido por alguien que posee la clave correspondiente y que no ha sido modificado desde la firma. No demuestra que el contenido sea secreto, que el usuario siga activo, que los permisos no hayan cambiado o que el token no haya sido robado. Todas esas propiedades requieren decisiones adicionales.

La expiración limita el tiempo durante el cual una credencial robada puede ser útil. El refresh reduce la necesidad de access tokens largos, pero introduce una credencial de mayor valor que debe protegerse con más cuidado. La rotación de refresh tokens permite detectar reutilización y reducir persistencia de una credencial comprometida.

El experimento de modificar el payload es especialmente importante. Si cambiamos sub o un rol y sólo volvemos a codificar Base64URL, el token sigue teniendo una apariencia correcta y se puede decodificar. La verificación criptográfica, sin embargo, debe rechazarlo. Esa diferencia es el núcleo conceptual del módulo.

Preguntas de control

- ¿Por qué Base64URL no ofrece confidencialidad?
- ¿Qué propiedad se rompe al modificar el payload sin recalcular la firma?
- ¿Qué ventaja tiene un access token corto combinado con refresh frente a un access token de varios días?

6.6 - Generación y validación de JWT con JWT

6.6.1 JWT

6.6.1.1 Qué es JWT y por qué usarla

JWT es una biblioteca Java para crear, firmar, analizar y validar JWT. Evita implementar a mano detalles delicados de Base64URL, serialización de claims y verificación de firmas.

M6 utiliza JWT 0.13.0. La versión importa porque la API ha evolucionado y muchos ejemplos antiguos de Internet usan métodos obsoletos. El código del curso adopta la API actual.

6.6.1.2 Los módulos de JWT

JWT se distribuye en varios artefactos: una API pública y módulos de implementación/JSON que se usan en runtime. Maven debe declarar correctamente las dependencias para compilar contra la API y disponer de la implementación al ejecutar.

Separar los módulos evita acoplar el código de aplicación a clases internas y permite a la biblioteca elegir el soporte JSON adecuado.

6.6.1.3 La API de JWT

La creación se hace con `Jwts.builder()` y el análisis con un parser configurado con la clave de verificación. El estado final utiliza `Jwts.SIG.HS256` de forma explícita y `signWith(clave, Jwts.SIG.HS256)`.

Esta explicitud elimina ambigüedades y hace visible la política criptográfica del proyecto.

6.6.2 Clave criptográfica

6.6.2.1 Cómo se representa una clave en JWT

Para HMAC JWT trabaja con un objeto `SecretKey`, no con un `String` utilizado directamente como contraseña. La configuración decodifica un secreto Base64 y construye una clave adecuada para HS256.

Representar la clave como bytes y luego como `SecretKey` evita errores de encoding y permite validar su tamaño antes de usarla.

6.6.2.2 Generar una clave segura

Una clave HMAC debe contener suficiente entropía. No sirven palabras legibles, nombres de proyecto ni secretos cortos. Para desarrollo puede fijarse un valor de prueba largo; en producción debe proceder de una variable de entorno o gestor de secretos.

La clave no se registra en logs ni se devuelve en respuestas. Su compromiso permitiría fabricar tokens válidos.

6.6.2.3 Construir la clave en el código

`JwtConfig` concentra la lectura de propiedades y la construcción de `SecretKey`. Así `JwtService` recibe una dependencia ya válida y no repite lógica de configuración.

Esta separación también facilita tests: podemos inyectar una clave controlada para verificar generación y expiración sin depender del entorno del desarrollador.

6.6.3 JwtService

6.6.3.1 La estructura del servicio

`JwtService` centraliza generar, parsear y validar tokens. No debe conocer HTTP ni decidir roles de endpoints. Sus entradas son identidades/claims y sus salidas son tokens o datos verificados.

Concentrar la lógica evita que login, refresh y filtro implementen variantes ligeramente distintas de la misma validación.

6.6.3.2 Generar el token de acceso

El access token incluye subject, fechas de emisión/expiración y los claims que el diseño necesite. Se firma con HS256 y la clave configurada. La duración proviene de `jwt.expiration`, no de un número mágico repetido.

Un token generado se considera válido sólo si más tarde el parser puede verificar firma y expiración. Los tests de `JwtService` deben cubrir ambas cosas.

6.6.3.3 Generar el refresh token

El refresh token se firma igual, pero tiene una vida distinta y un propósito específico. Conviene identificar su tipo o gestionar su uso en un servicio de revocación para impedir que se utilice como access token por accidente.

M6 implementa rotación: al refrescar se invalida el refresh anterior y se entrega uno nuevo. Esto reduce la ventana de reutilización.

6.6.4 Login

6.6.4.1 El flujo de login

El cliente envía username y password al endpoint de login. AuthenticationManager valida las credenciales contra UserDetailsService y PasswordEncoder. Sólo después del éxito se generan access y refresh token.

Separar autenticación y emisión de token es importante: un JWT no “comprueba” la contraseña; simplemente representa una identidad que ya fue autenticada.

6.6.4.2 El AuthenticationManager en el login

AuthenticationManager recibe un UsernamePasswordAuthenticationToken no autenticado. Si el proveedor acepta las credenciales, devuelve otro objeto autenticado con el principal y sus authorities.

El servicio usa ese resultado como fuente de verdad para emitir tokens. No busca al usuario por su cuenta y compara la contraseña manualmente.

6.6.4.3 El DTO de respuesta del login

La respuesta expone access_token, refresh_token, token_type y expires_in. Un DTO explícito estabiliza el contrato JSON y evita devolver objetos internos de Spring Security.

token_type vale Bearer, lo que indica al cliente cómo usar el access token en la cabecera Authorization.

6.6.5 Refresh y seguridad operacional

6.6.5.1 El endpoint de refresh

El refresh recibe un refresh token, comprueba que sea válido y utilizable, lo consume/rota y devuelve un nuevo par. No vuelve a pedir contraseña porque el refresh token es precisamente la credencial de renovación.

Si el token está expirado, revocado o ya usado, la operación debe fallar. No debe “arreglarse” aceptándolo de todas formas.

6.6.5.2 Buenas prácticas con tokens

Los errores de tokens se traducen a respuestas coherentes, sin exponer la clave ni detalles criptográficos. Las duraciones se externalizan y el perfil de producción exige el secreto desde JWT_SECRET.

También conviene mantener los access tokens cortos y reservar el estado de revocación para los casos donde aporta valor, como logout y refresh de un solo uso.

6.6.5.3 Errores comunes

Entre los errores comunes están usar dependencias JWT de versiones mezcladas, firmar con una clave String insuficiente, usar un algoritmo implícito, dejar un secreto de producción con fallback y no distinguir access de refresh.

Los tests de integración de JwtService son el lugar adecuado para capturar estos fallos antes de probar toda la API.

Generación con JJWT 0.13.0

```
String token = Jwts.builder()  
    .subject(username)  
    .issuedAt(Date.from(ahora))  
    .expiration(Date.from(ahora.plusMillis(expirationMs)))  
    .signWith(clave, Jwts.SIG.HS256)  
    .compact();
```

En producción la clave se obtiene de JWT_SECRET; no se incluye una alternativa insegura en el propio fichero de configuración.

Profundización: separar criptografía, autenticación y ciclo de tokens

JwtService no debe convertirse en un “servicio de seguridad total”. Su responsabilidad es crear y validar tokens. AuthService coordina credenciales, login, refresh y logout. JwtAuthenticationFilter adapta un token válido a la infraestructura HTTP. Esta separación permite probar cada pieza con escenarios pequeños y evita que una excepción criptográfica termine mezclada con lógica de roles o persistencia.

La configuración de claves también merece una frontera propia. En desarrollo podemos utilizar un secreto controlado para reproducir tests, pero producción debe exigir una variable externa. Un fallback cómodo en application-prod.properties convierte un error de despliegue en una vulnerabilidad silenciosa; por eso el perfil productivo del curso no ofrece alternativa.

El refresh token no se valida únicamente con la misma condición que el access token. Ambos pueden estar correctamente firmados y no por ello ser intercambiables. La aplicación necesita distinguir propósito y uso, además de comprobar expiración y revocación.

Preguntas de control

- ¿Por qué el login debe delegar la contraseña en AuthenticationManager?
- ¿Qué problema aparece si un access token puede enviarse al endpoint de refresh?
- ¿Por qué una clave Base64 debe decodificarse antes de construir SecretKey?

6.7 - Filtro JWT y reconstrucción de la identidad

6.7.1 El problema de la autenticación con JWT

6.7.1.1 El problema de la autenticación con JWT

Después del login el cliente ya no envía username y password en cada llamada. Envía un Bearer token. La aplicación necesita convertir ese token en una identidad que Spring Security pueda reconocer antes de evaluar cualquier regla de acceso.

Ese trabajo se realiza en un filtro. El filtro valida la credencial y, si es correcta, construye un Authentication que se almacena en el SecurityContext. A partir de ese punto, controladores y expresiones de autorización trabajan igual que con cualquier otro mecanismo de autenticación.

6.7.1.2 Por qué OncePerRequestFilter

OncePerRequestFilter garantiza una ejecución controlada por petición y ofrece una base adecuada para filtros propios de Spring. Evita tener que gestionar directamente detalles del ciclo servlet y hace más clara la implementación de doFilterInternal.

El filtro debe ser pequeño: extraer credencial, validarla, reconstruir identidad y continuar la cadena. La lógica de negocio no pertenece aquí.

6.7.1.3 El flujo completo de una petición con JWT

La secuencia es: llega la petición, el filtro busca Authorization, extrae el Bearer token, valida firma y expiración, obtiene el username, carga el usuario, crea un UsernamePasswordAuthenticationToken autenticado y lo guarda en SecurityContextHolder. Después llama a filterChain.doFilter.

Si no hay token, el filtro no inventa una identidad; deja que la cadena continúe y que la política de seguridad decida si la ruta admite anónimo o exige autenticación.

6.7.2 Cabecera Authorization

6.7.2.1 La cabecera Authorization

La forma estándar es Authorization: Bearer <token>. El prefijo y el espacio son parte del formato. El filtro debe comprobarlos antes de intentar parsear la cadena.

No conviene aceptar tokens desde parámetros query porque pueden quedar registrados en historiales, proxies o logs. La cabecera mantiene la credencial en el canal previsto por HTTP.

6.7.2.2 Qué hacer si no hay token

La ausencia de token no es automáticamente un error del filtro. Una ruta pública puede funcionar perfectamente sin él. Por eso el filtro simplemente continúa cuando la cabecera no existe o no empieza por Bearer.

Si la ruta es protegida, la fase de autorización detectará que no existe autenticación y el AuthenticationEntryPoint responderá 401. Esta separación evita codificar el mapa de rutas dentro del filtro.

6.7.2.3 No usar el filtro para autorizar

El filtro autentica, no decide si USER puede entrar en /admin. Las reglas de autorización deben permanecer en SecurityFilterChain o en @PreAuthorize.

Mezclar ambas responsabilidades convierte el filtro en un conjunto de if por URL y hace muy difícil mantener la política. El filtro sólo establece una identidad fiable.

6.7.3 Authentication y principal

6.7.3.1 Qué es un Authentication

Authentication representa la identidad de seguridad dentro de Spring. Antes de validar credenciales puede ser una solicitud no autenticada; después contiene principal, authorities y el indicador de autenticación.

En JWT no reutilizamos la credencial como contraseña. Una vez verificado el token, construimos un Authentication autenticado con el principal y sus authorities para el resto de la petición.

6.7.3.2 Usar UsuarioPrincipal como principal

UsuarioPrincipal adapta la entidad Usuario al contrato de Spring Security y puede exponer identificador, username, email y authorities sin revelar la contraseña. Esto mejora endpoints de perfil y expresiones SpEL que necesitan el id del usuario.

El principal debe ser inmutable desde el punto de vista del request. Cambiar roles en memoria durante una petición no debe sustituir la fuente persistente de verdad.

6.7.3.3 El UsuarioDetailsService en el flujo

Después de extraer el username del token, el filtro carga la identidad actual con UsuarioDetailsService. Esto permite usar los roles vigentes de la base de datos en lugar de confiar ciegamente en datos antiguos almacenados en el token.

La decisión también simplifica el diseño: el token identifica al usuario y la aplicación reconstruye el principal con la misma lógica utilizada por el login.

6.7.4 Registro del filtro

6.7.4.1 Añadir el filtro a la cadena

El filtro se registra con `addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`. Así la identidad JWT existe antes de que las reglas de autorización necesiten el contexto.

Registrar un filtro no basta: también debemos retirar HTTP Basic del estado final para que la API no acepte dos mecanismos diferentes sin intención.

6.7.4.2 El filtro como bean

`JwtAuthenticationFilter` es un componente administrado por Spring y recibe `JwtService`, `UsuarioDetailsService` y, cuando procede, el servicio de revocación. La inyección por constructor hace visibles sus dependencias y facilita tests.

No debe crear repositorios o parsers por su cuenta. La configuración y validación de tokens pertenecen a servicios dedicados.

6.7.4.3 Orden de los filtros

Un orden incorrecto puede producir 401 aunque el token sea correcto o duplicar autenticaciones. La referencia a `UsernamePasswordAuthenticationFilter` ofrece un punto conocido en la cadena.

También conviene evitar volver a autenticar si `SecurityContextHolder` ya contiene una identidad válida. El filtro puede comprobarlo antes de cargar de nuevo el usuario.

6.7.5 Diagnóstico y buenas prácticas

6.7.5.1 Depurar el filtro

Para depurar, observa tres datos: si la cabecera se detecta, si el parser valida el token y si el `SecurityContext` queda poblado. Los logs de DEBUG pueden ayudar, pero nunca deben imprimir el token completo ni la clave.

Un test de integración que accede a una ruta protegida con token real proporciona una evidencia más fiable que inspeccionar logs manualmente.

6.7.5.2 Errores comunes

Los errores habituales son cortar la cadena cuando no hay token, devolver 401 directamente desde el filtro para cualquier problema, olvidar el prefijo Bearer, cargar el usuario antes de validar la firma o registrar el filtro después del punto donde se necesita la autenticación.

También es un error autorizar por URL dentro del filtro; esa lógica debe permanecer declarativa.

6.7.5.3 Buenas prácticas

El filtro debe ser determinista, no conservar estado por petición y delegar criptografía en JwtService. Las excepciones esperables se traducen de forma coherente y las rutas públicas siguen funcionando sin credencial.

La responsabilidad puede resumirse en una frase: **si existe un token válido, establecer una identidad; en cualquier otro caso, no fabricar permisos.**

Esqueleto del filtro

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    filterChain.doFilter(request, response);
    return;
}
String token = header.substring(7);
String username = jwtService.extraerUsername(token);
UserDetails user = userDetailsService.loadUserByUsername(username);
Authentication auth = new UsernamePasswordAuthenticationToken(
    user, null, user.getAuthorities());
SecurityContextHolder.getContext().setAuthentication(auth);
filterChain.doFilter(request, response);
```

La implementación real añade validación, revocación y manejo cuidadoso de errores.

Profundización: un filtro robusto hace poco, pero lo hace bien

El diseño de un buen filtro se caracteriza por sus límites. No consulta permisos por URL, no ejecuta reglas de negocio y no genera respuestas de éxito. Su función es observar una credencial en la petición y, cuando esa credencial es válida, preparar el contexto para que otros componentes tomen decisiones.

Esto también define cómo debe comportarse ante ausencia de token. Si la ruta es pública, la petición tiene que continuar. Si la ruta es privada, la ausencia será detectada posteriormente por la política. En cambio, cuando existe una credencial explícita pero inválida, conviene impedir que produzca una identidad parcial. El SecurityContext debe permanecer limpio.

La depuración es más sencilla si se piensa en estados: cabecera ausente, cabecera mal formada, token inválido, token válido pero usuario inexistente, token válido y principal construido. Cada estado puede convertirse en un test independiente.

Preguntas de control

- ¿Por qué el filtro no debería devolver 403?
- ¿Qué diferencia hay entre “no hay token” y “hay token inválido”?
- ¿Por qué el filtro se registra antes de UsernamePasswordAuthenticationFilter?

6.8 - Configuración final stateless, errores y autorización completa

6.8.1 SecurityFilterChain final

6.8.1.1 El SecurityFilterChain con JWT

El estado final reúne todas las decisiones acumuladas: CSRF desactivado para la API Bearer, CORS habilitado, sesiones STATELESS, allowlist pública explícita, reglas por rol, manejadores de 401/403 y filtro JWT antes del filtro de usuario/contraseña.

La configuración ya no incluye HTTP Basic. Si una petición protegida no aporta un Bearer token válido, no existe un mecanismo alternativo silencioso que la autentique.

6.8.1.2 Por qué personalizar los manejadores de error

Los fallos de seguridad ocurren antes del controlador, de modo que un `@RestControllerAdvice` convencional no siempre puede construir la respuesta. `AuthenticationEntryPoint` atiende fallos de autenticación y `AccessDeniedHandler` atiende denegaciones de autorización.

Personalizarlos permite conservar el contrato `ErrorResponse` de M5, incluido `traceId`, en lugar de devolver formatos distintos según la capa que produjo el error.

6.8.1.3 Estructura del `JwtAuthenticationEntryPoint`

El entry point escribe una respuesta 401 con `Content-Type JSON` y el mismo esquema de error usado por el resto de la API. No revela si el username existe ni detalles de la firma.

La escritura se centraliza en `SecurityErrorWriter` para que 401 y 403 compartan estructura y no dupliquen serialización.

6.8.2 Mapa de endpoints

6.8.2.1 Clasificación de endpoints

Antes de escribir matchers conviene clasificar rutas: públicas, autenticadas, de gestión y administrativas. La clasificación evita reglas ad hoc que se acumulan sin una política general.

En M6 sólo son públicas las operaciones de registro/login/refresh declaradas, la zona `/api/v1/public/**`, H2 en desarrollo y la documentación OpenAPI/Swagger. El resto requiere identidad.

6.8.2.2 Orden de las reglas

Spring Security evalúa matchers en orden. Una regla amplia como `/**.permitAll()` colocada antes convertiría en irrelevantes las restricciones posteriores.

Por eso se escriben primero las excepciones públicas y las zonas con roles específicos, y se termina con `anyRequest().authenticated()`. El último matcher actúa como red de seguridad.

6.8.2.3 Endpoints con permisos específicos

Las zonas `/api/v1/admin/**` exigen ADMIN y `/api/v1/gestor/**` admiten GESTOR o ADMIN. Otras reglas más cercanas al negocio pueden añadirse con `@PreAuthorize`.

Esta división muestra dos niveles: perímetro HTTP y autorización del caso de uso. Ambos se prueban de forma independiente.

6.8.3 Roles con JWT

6.8.3.1 Los roles vienen del token

En una implementación posible, los roles podrían viajar como claims. En M6, sin embargo, el token identifica al usuario y el filtro reconstruye el principal desde base de datos, de manera que las authorities actuales se obtienen del sistema.

Esto evita que un token conserve durante demasiado tiempo un rol retirado en la base de datos. La contrapartida es una consulta adicional para reconstruir identidad.

6.8.3.2 hasRole y hasAuthority con JWT

Una vez creado el Authentication, la autorización no necesita saber que la identidad llegó por JWT. hasRole, hasAuthority y @PreAuthorize trabajan exactamente con las authorities del contexto.

Ésta es una de las ventajas de integrarse correctamente con Spring Security: el mecanismo de autenticación y la política de autorización quedan desacoplados.

6.8.3.3 Combinar filtro, URL y método

El filtro valida credenciales, la cadena HTTP controla el perímetro y la seguridad por método protege casos de uso concretos. Si cada capa respeta esa responsabilidad, el diseño sigue siendo comprensible incluso cuando crece.

Los tests finales deben demostrar que estas capas cooperan: token válido sin rol produce 403, token válido con rol produce éxito y token inválido no crea autenticación.

6.8.4 Flujo completo y logout

6.8.4.1 Diagrama del flujo

Puede leerse el flujo de izquierda a derecha: cliente -> filtro JWT -> SecurityContext -> matchers HTTP -> proxy de método -> controlador/servicio. Una respuesta de seguridad puede producirse en varias etapas, pero siempre antes de ejecutar una operación no autorizada.

Esta visión evita depurar sólo el controlador cuando el rechazo ocurrió mucho antes.

6.8.4.2 Manejo de la expiración en el cliente

El servidor debe rechazar un access token expirado. El cliente, al recibir 401 por expiración, puede usar el refresh token para renovar y repetir la operación. La lógica de renovación pertenece al cliente y al endpoint de refresh, no al filtro de cada recurso.

Un bucle infinito de refresh es un error de cliente; si el refresh también falla, la sesión lógica del usuario debe terminar.

6.8.4.3 Logout con JWT

Como no existe sesión HTTP que destruir, logout significa impedir que determinadas credenciales sigan siendo aceptadas. M6 mantiene una revocación didáctica en memoria y consume refresh tokens de un solo uso.

En sistemas distribuidos esta estrategia debería externalizarse o sustituirse por tokens muy cortos y un mecanismo de revocación compartido. El módulo documenta claramente el alcance de la implementación.

6.8.5 Diagnóstico y buenas prácticas

6.8.5.1 Diagnóstico de problemas comunes

Ante un 401 inesperado, comprueba cabecera Bearer, firma, expiración, revocación y creación del SecurityContext. Ante un 403, la autenticación probablemente existe y el siguiente paso es revisar authorities y matchers.

Separar ambos diagnósticos ahorra tiempo. Un 403 no se corrige regenerando el mismo token si el usuario sigue sin rol.

6.8.5.2 Errores comunes

Es fácil dejar `httpBasic`, usar `permitAll` demasiado amplio, crear sesión accidentalmente, tratar 403 como 401 o permitir cualquier ruta bajo `/auth/**`. También puede olvidarse CORS al reconfigurar `HttpSecurity`.

El estado final evita estos atajos y conserva la política CORS heredada mediante `cors(Customizer.withDefaults)`.

6.8.5.3 Buenas prácticas

Una configuración segura debe ser explícita, mínima y testeable. Las rutas públicas se enumeran, las credenciales de producción se externalizan, la sesión está desactivada y los errores usan un formato uniforme.

Además, cualquier simplificación didáctica —como la revocación en memoria— debe quedar identificada como tal y no presentarse como arquitectura distribuida de producción.

Configuración final resumida

http

```
.csrf(AbstractHttpConfigurer::disable)
.cors(Customizer.withDefaults())
.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.POST,
        "/api/v1/auth/registro", "/api/v1/auth/login", "/api/v1/auth/refresh").permitAll()
    .requestMatchers("/api/v1/public/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/v1/gestor/**").hasAnyRole("GESTOR", "ADMIN")
    .anyRequest().authenticated())
.exceptionHandling(ex -> ex
    .authenticationEntryPoint(entryPoint)
    .accessDeniedHandler(deniedHandler))
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

Profundización: cerrar la arquitectura sin dejar puertas laterales

La fase final consiste tanto en añadir piezas como en retirar las temporales. HTTP Basic debe desaparecer, la sesión debe quedar en modo STATELESS y las rutas públicas deben ser una allowlist concreta. Dejar cualquiera de esas decisiones a medias puede crear un segundo camino de autenticación o una superficie pública mayor de la prevista.

Los errores 401 y 403 forman parte del contrato de la API. Un cliente debe poder procesarlos con la misma estructura que otros errores, y los logs internos pueden conservar más detalle que el cuerpo HTTP. Separar mensaje público y diagnóstico interno evita revelar información sobre usernames, firmas o claves.

CORS sigue siendo relevante aunque la API use JWT. El navegador aplica la política de origen antes de que la aplicación pueda aprovechar el token, y las peticiones preflight deben recibir una configuración coherente. Integrar `cors(Customizer.withDefaults())` permite reutilizar `CorsConfig` en lugar de crear una política paralela dentro de seguridad.

Preguntas de control

- ¿Por qué STATELESS no elimina la necesidad de revocar refresh tokens?
- ¿Qué riesgo tiene `requestMatchers("/api/v1/auth/**").permitAll()` frente a una allowlist explícita?
- ¿Qué información debe compartir un 401 con el cliente y qué información debe quedar sólo en logs?

6.9 - Pruebas de seguridad

6.9.1 La seguridad es funcionalidad

6.9.1.1 La seguridad es funcionalidad

Una política de acceso produce resultados observables y, por tanto, debe probarse como cualquier otra funcionalidad. El hecho de que una anotación esté presente no demuestra que esté activa ni que el matcher tenga el orden correcto.

Los tests convierten la política en una especificación ejecutable: quién puede llamar, con qué credencial y qué status debe recibir.

6.9.1.2 Qué se puede testear

Podemos probar autenticación, autorización, formato de errores, login, refresh, logout, filtro JWT, rutas públicas y comportamiento de usuarios concretos. También podemos comprobar que un token manipulado o expirado no crea identidad.

La cobertura útil no consiste en contar anotaciones, sino en recorrer caminos de éxito y rechazo representativos.

6.9.1.3 Herramientas de testing de seguridad

Spring Security Test se integra con MockMvc y el contexto de pruebas. Ofrece anotaciones de usuario simulado, post-processors para construir autenticaciones y matchers específicos de seguridad.

Para los flujos JWT reales, complementaremos esos helpers con peticiones al login y uso de los tokens generados por la aplicación.

6.9.2 Usuarios simulados

6.9.2.1 @WithMockUser

@WithMockUser crea una identidad sintética para el test. Es ideal para probar políticas de roles sin depender de la base de datos. Podemos declarar username, roles o authorities y verificar rápidamente 200/403.

No demuestra que UsuarioDetailsService cargue correctamente un usuario real. Por eso se combina con otros tipos de prueba.

6.9.2.2 @WithUserDetails

@WithUserDetails pide al UserDetailsService del contexto que cargue una identidad concreta. Resulta útil cuando queremos verificar la integración con usuarios persistidos o con un servicio real de identidades.

El usuario debe existir antes de que se prepare el contexto de seguridad del test, lo que exige cuidar la inicialización de datos.

6.9.2.3 @WithAnonymousUser

@WithAnonymousUser fuerza un escenario anónimo incluso si la clase de test define otra identidad por defecto. Permite documentar que una ruta pública funciona o que una ruta protegida rechaza explícitamente el acceso sin credenciales.

Los casos anónimos son esenciales para detectar permitAll accidentales.

6.9.3 Post-processors

6.9.3.1 Post-processors para peticiones

MockMvc permite añadir autenticación directamente a una petición mediante post-processors. Esto resulta más flexible que una anotación de método cuando diferentes requests del mismo test necesitan identidades distintas.

También facilita construir matrices de permisos sin duplicar clases de test.

6.9.3.2 El post-processor jwt()

En aplicaciones que usan el soporte OAuth2 de Spring Security existe un post-processor `jwt()` que simula un JWT y sus claims. Nuestro proyecto implementa un filtro JWT propio, por lo que la prueba más representativa del filtro usa tokens generados por `JwtService`.

Aun así, conocer `jwt()` ayuda a distinguir pruebas de autorización basada en claims de pruebas del parser/filtro real.

6.9.3.3 Verificaciones con `SecurityMockMvcResultMatchers`

Los matchers de seguridad permiten comprobar si una petición terminó autenticada, anónima o con una authority concreta. Combinados con `status()` y aserciones JSON, proporcionan una especificación precisa.

La verificación no debe limitarse al status: cuando sea relevante también comprobaremos cuerpo de error, `traceId` y datos del perfil autenticado.

6.9.4 Tests de flujos

6.9.4.1 Tests de autenticación (login)

El test de login envía credenciales válidas y espera un par de tokens con tipo Bearer y expiración. También debe enviar una contraseña errónea y comprobar que no se emite ningún token.

Esto prueba la integración de `AuthenticationManager`, usuarios persistidos, password encoder y emisión JWT.

6.9.4.2 Tests de refresh y logout

Refresh debe aceptar un token utilizable, rotarlo y rechazar su reutilización. Logout debe provocar que las credenciales revocadas dejen de aceptarse según la estrategia didáctica del módulo.

Estos escenarios son importantes porque el camino feliz de login no revela errores de ciclo de vida.

6.9.4.3 Tests de autorización

La autorización se prueba con identidades que difieren sólo en sus roles. Un USER y un ADMIN pueden estar correctamente autenticados, pero obtener respuestas distintas al llamar a una operación administrativa.

Ese contraste demuestra que 403 pertenece a autorización, no a autenticación.

6.9.5 Buenas prácticas de tests

6.9.5.1 Buenas prácticas con tests de seguridad

Los tests deben ser pequeños, nombrar claramente el escenario y evitar depender unos de otros. Los datos se preparan de forma idempotente y cada prueba expresa su propia identidad.

Separar pruebas de política, de servicios JWT y de flujo end-to-end facilita diagnosticar el origen de un fallo.

6.9.5.2 Errores comunes

Son comunes los falsos positivos causados por desactivar filtros en un `@WebMvcTest`, declarar un `@MockBean` que ya no corresponde a la versión actual de Spring Boot o contar tests sin leer el resultado de Surefire.

En M6 los slices MVC que preservan comportamiento heredado desactivan filtros cuando su objetivo no es seguridad, pero proporcionan mocks para las dependencias de seguridad que Spring aún necesita instanciar.

6.9.5.3 Estructura de un test de seguridad completo

Un test completo prepara datos, obtiene o construye la identidad, ejecuta la petición, comprueba status y cuerpo, y verifica el post-estado cuando la operación modifica datos. Si prueba JWT real, genera el token con el mismo `JwtService` de la aplicación.

El módulo termina cuando esta batería demuestra conjuntamente autenticación, autorización, errores y continuidad con los módulos anteriores.

Ejemplo con MockMvc

```
mockMvc.perform(get("/api/v1/admin/usuarios")
    .with(user("ana").roles("USER")))
    .andExpect(status().isForbidden());

mockMvc.perform(get("/api/v1/admin/usuarios")
    .with(user("admin").roles("ADMIN")))
    .andExpect(status().isOk());
```

La misma política debe verificarse también en el flujo JWT real, no sólo con identidades simuladas.

Profundización: construir una suite que detecte regresiones reales

Una buena suite de seguridad mezcla distintos niveles. Los tests de servicio prueban generación y validación de tokens con precisión. Los tests de controlador comprueban políticas con identidades simuladas. Los tests de integración ejecutan el filtro real, la base de datos y el flujo de login para detectar errores de ensamblaje que los mocks no ven.

Los slices MVC requieren especial atención desde que la aplicación incorpora seguridad. Desactivar filtros con `@AutoConfigureMockMvc(addFilters = false)` evita que un test de controlador heredado se transforme de repente en un test de autenticación, pero el contexto puede seguir necesitando beans usados por los filtros durante la construcción. En Spring Boot 3.5 se utiliza `@MockitoBean` para proporcionar esas dependencias cuando corresponde.

El número total de tests es un indicador de que se ha ejecutado la suite esperada, pero no sustituye la lectura de resultados. Surefire debe informar cero fallos y cero errores. El runtime final completa la evidencia con peticiones reales a la aplicación arrancada.

Preguntas de control

- ¿Qué diferencia demuestra un test con `@WithMockUser` y un test que obtiene un JWT mediante login?
- ¿Por qué conviene conservar tests heredados que no están relacionados directamente con seguridad?
- ¿Qué falso positivo puede producir un slice MVC configurado de forma incorrecta?

Cierre del módulo

La aplicación final de M6 ya no depende de una sesión HTTP ni de HTTP Basic. El login valida credenciales persistidas, emite access y refresh token, el filtro reconstruye la identidad en cada petición, las reglas de autorización actúan sobre roles reales y los errores de seguridad usan el mismo contrato JSON que el resto de la API. Los tests cubren tanto los caminos de éxito como los rechazos 401/403 y el ciclo de refresh/logout.

La idea que debe quedar al terminar es que la seguridad no es un añadido aislado: forma parte del contrato de la API y debe diseñarse, observarse y probarse con el mismo rigor que persistencia o reglas de negocio.